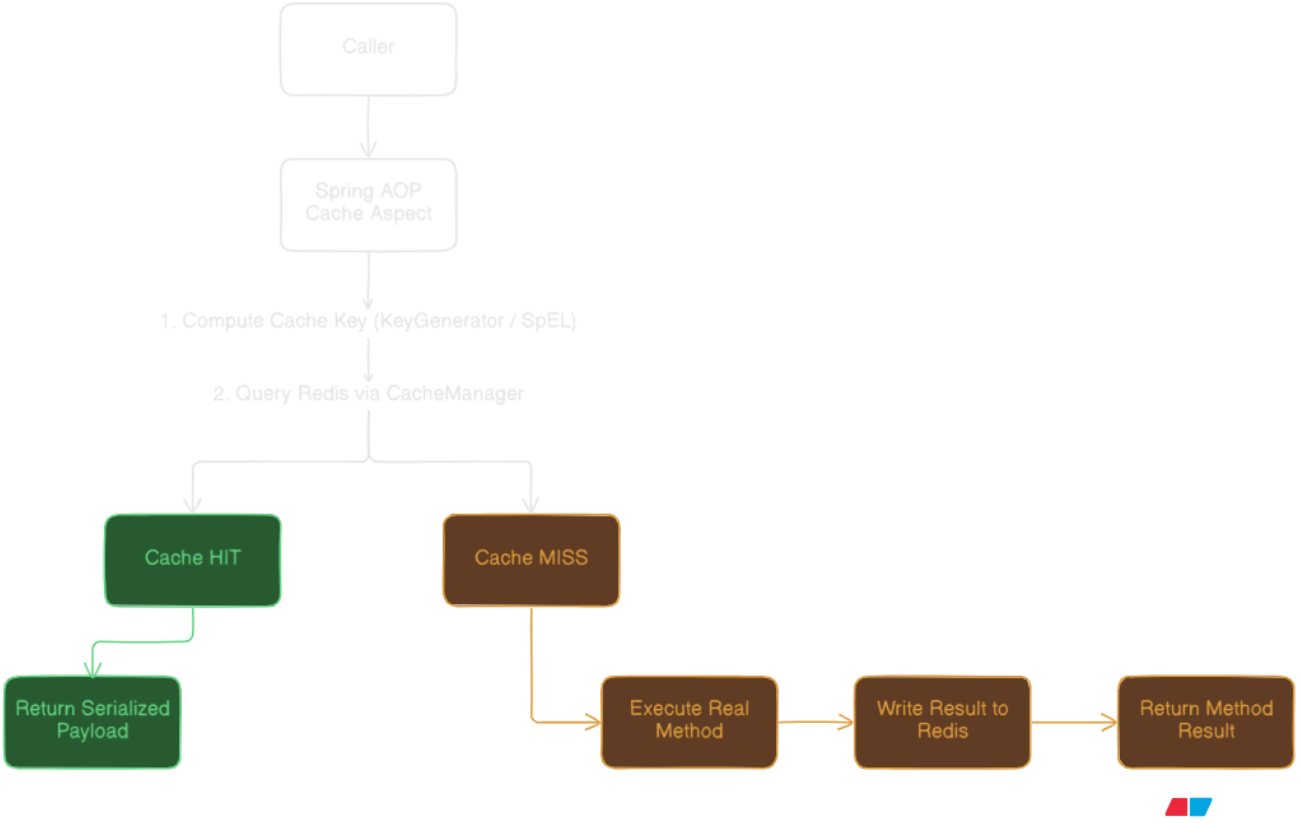
# Understanding Cacheable & SpEL

## 1. Spring AOP Proxy Mechanics

When you annotate a service method with `@Cacheable`, Spring does not modify your compiled bytecode directly. Instead, it wraps your bean inside a **Dynamic Proxy** (CGLIB or JDK Dynamic Proxy).




### What happens under the hood?

1. The caller invokes the method on the **proxy instance**, not the target bean instance.
2. The proxy checks the configured `CacheManager` (e.g., `RedisCacheManager`).
3. If the key exists in Redis (**Cache Hit**), the target method is skipped, and the deserialized object is returned immediately.
4. If the key does not exist (**Cache Miss**), the proxy delegates execution to the target method, receives the result, writes it to Redis, and returns it to the caller.

## 2. The Internal Call (Self-Invocation) Problem

One of the most common production bugs in Spring Caching is calling a cached method from within the same service class.

### The Broken Pattern

```java
@Service
public class FinancialSummaryService {

    // Internal caller
    public FinancialSummaryDTO getSummaryForDashboard(Long userId) {
        // Direct internal call! Bypass proxy!
        return fetchAggregatedSummary(userId);
    }

    @Cacheable(value = "financialSummaries", key = "#userId")
    public FinancialSummaryDTO fetchAggregatedSummary(Long userId) {
        // Heavy DB query + Redis calculation
        return expensiveDatabaseCalculation(userId);
    }
}
```

## Why it fails

- When `getSummaryForDashboard()` calls `fetchAggregatedSummary(userId)`, it uses the implicit `this` reference (`this.fetchAggregatedSummary(userId)`)
- `this` points to the **underlying target instance**, not the **Spring AOP Proxy**. Because the call does not pass through the proxy, the `@Cacheable` annotation is completely ignored, executing the expensive method on every request.

## The Solutions

### Solution A: Architectural Refactoring (Recommended)

Separate the caching target into a distinct bean to enforce proxy routing through Spring's ApplicationContext.

```java
@Service
@RequiredArgsConstructor
public class FinancialSummaryService {

    private final FinancialSummaryCacheService cacheService;

    public FinancialSummaryDTO getSummaryForDashboard(Long userId) {
        // Routes through Spring Proxy correctly
        return cacheService.fetchAggregatedSummary(userId);
    }
}

@Service
public class FinancialSummaryCacheService {

    @Cacheable(value = "financialSummaries", key = "#userId")
```

```
    public FinancialSummaryDTO fetchAggregatedSummary(Long userId) {
        return expensiveDatabaseCalculation(userId);
    }
}
```

### Solution B: Self-Autowiring via `@Lazy`

If splitting the service adds unnecessary overhead, inject the proxy into itself.

```
@Service
public class FinancialSummaryService {

    @Autowired
    @Lazy
    private FinancialSummaryService self;

    public FinancialSummaryDTO getSummaryForDashboard(Long userId) {
        // Invokes method on the injected proxy instance
        return self.fetchAggregatedSummary(userId);
    }

    @Cacheable(value = "financialSummaries", key = "#userId")
    public FinancialSummaryDTO fetchAggregatedSummary(Long userId) {
        return expensiveDatabaseCalculation(userId);
    }
}
```

## 3. SpEL Deep Dive: Keys, Conditions, and Filters

Spring Expression Language (SpEL) provides runtime contextual evaluation for cache keys and execution rules.

### Context Variables Available in SpEL

| Variable | Description | Example |
|---|---|---|
| `#root.methodName` | Target method name | `#root.methodName` |
| `#root.args[n]` | Arguments array by index | `#root.args[0]` |
| `#paramName` | Method argument by parameter name | `#userId` |
| `#result` | Returned object (**`unless` condition only**) | `#result == null` |

### A. Composite Keys

Never rely on default key generation when a method accepts multiple parameters or complex DTOs. Build explicit composite keys to avoid key collisions in Redis.

```
@Cacheable(
    value = "userTransactions",
    key = "#userId + ':' + #category + ':' + #pageable.pageNumber"
)
public Page<TransactionDTO> getTransactions(Long userId, String category,
Pageable pageable) {
    return transactionRepository.findByUserAndCategory(userId, category,
pageable);
}
```

*Generated Redis Key:* `userTransactions::1001:INVESTMENTS:0`

### B. Conditional Caching ( `condition` vs `unless` )

- `condition` **(Pre-execution evaluation):** Determines if Redis should be inspected or updated *before* executing the method body. `#result` is **not** available here.
- `unless` **(Post-execution evaluation):** Evaluates *after* the method body executes. If the condition is true, the result is **not cached**. `#result` is available here.

```
@Cacheable(
    value = "financialSummaries",
    key = "#userId",
    // Only attempt caching for non-system/VIP user IDs
    condition = "#userId > 1000",
    // Do NOT cache if result is null or status is PENDING
    unless = "#result == null || #result.status == 'PENDING'"
)
public FinancialSummaryDTO getSummary(Long userId) {
    return calculateSummary(userId);
}
```

## 4. Step-by-Step Implementation

### Step 1: Redis Cache Configuration

Set up the `RedisCacheManager` using `GenericJackson2JsonRedisSerializer` for human-readable JSON values in Redis.

```
@Configuration
@EnableCaching
public class RedisConfig {

    @Bean
    public RedisCacheManager cacheManager(RedisConnectionFactory
connectionFactory) {
        ObjectMapper objectMapper = new ObjectMapper();
        objectMapper.registerModule(new JavaTimeModule());
```

```
objectMapper.disable(SerializationFeature.WRITE_DATES_AS_TIMESTAMPS);

        GenericJackson2JsonRedisSerializer serializer =
                new GenericJackson2JsonRedisSerializer(objectMapper);

        RedisCacheConfiguration defaultConfig =
RedisCacheConfiguration.defaultCacheConfig()
                .entryTtl(Duration.ofMinutes(30))
                .disableCachingNullValues()
                .serializeKeysWith(

RedisSerializationContext.SerializationPair.fromSerializer(new
StringRedisSerializer())
                )
                .serializeValuesWith(

RedisSerializationContext.SerializationPair.fromSerializer(serializer)
                );

        return RedisCacheManager.builder(connectionFactory)
                .cacheDefaults(defaultConfig)
                .build();
    }
}
```

## Step 2: DTO Definition

```
public record FinancialSummaryDTO(
    Long userId,
    BigDecimal totalAssets,
    BigDecimal totalLiabilities,
    String status,
    LocalDateTime lastUpdated
) implements Serializable {}
```

## Step 3: Production-Grade Cache Service

```
@Service
@Slf4j
@RequiredArgsConstructor
public class AccountSummaryService {

    private final AccountRepository accountRepository;

    @Cacheable(
        value = "financialSummaries",
        key = "'user:' + #userId + ':type:' + #accountType",
```

```
        condition = "#userId != null",
        unless = "#result == null ||
#result.totalAssets().compareTo(T(java.math.BigDecimal).ZERO) == 0"
    )
    public FinancialSummaryDTO getAggregatedSummary(Long userId, String
accountType) {
        log.info("Cache miss! Computing financial summary for userId: {},
type: {}", userId, accountType);

        // Simulating heavy aggregation logic
        BigDecimal assets =
accountRepository.sumAssetsByUserIdAndType(userId, accountType);
        BigDecimal liabilities =
accountRepository.sumLiabilitiesByUserIdAndType(userId, accountType);

        if (assets == null) {
            return null; // 'unless' condition prevents caching null
results
        }

        return new FinancialSummaryDTO(
            userId,
            assets,
            liabilities,
            "ACTIVE",
            LocalDateTime.now()
        );
    }
}
```

# 5. Verification & Redis CLI Inspection

## Run Local Tests

Call the service twice from a controller or test context:

```
// Call 1: Triggers DB execution and stores in Redis
FinancialSummaryDTO res1 =
accountSummaryService.getAggregatedSummary(1001L, "SAVINGS");

// Call 2: Returns instantly from Redis
FinancialSummaryDTO res2 =
accountSummaryService.getAggregatedSummary(1001L, "SAVINGS");
```

## Inspect Redis state via `redis-cli`

Connect to your local Redis container:

```
docker exec -it redis redis-cli
```

Check the active key namespace:

```
127.0.0.1:6379> KEYS "financialSummaries*"
1) "financialSummaries::user:1001:type:SAVINGS"
```

Inspect value type and TTL:

```
127.0.0.1:6379> TYPE "financialSummaries::user:1001:type:SAVINGS"
string

127.0.0.1:6379> TTL "financialSummaries::user:1001:type:SAVINGS"
(integer) 1785
```

Read the JSON value payload directly:

```
127.0.0.1:6379> GET "financialSummaries::user:1001:type:SAVINGS"
"
{\"@class\":\"com.example.dto.FinancialSummaryDTO\",\"userId\":1001,\"tota
lAssets\":150000.50,\"totalLiabilities\":25000.00,\"status\":\"ACTIVE\",\"
lastUpdated\":[2026,9,7,11,30,0]}"
```

## 6. Common Production Problems

1. **Missing Default Constructor on DTOs:** When using Jackson without `JavaTimeModule` or Parameter Names module, record classes or immutable DTOs without default constructors can trigger `InvalidDefinitionException` during deserialization.
2. **Key Namespace Pollution:** Always use specific cache names in `@Cacheable(value = "...")` combined with domain-specific SpEL expressions ( `'user:' + #id` ). Avoid generic keys like `key = "#id"` across different entities to prevent data collisions.
3. **Mismatched Cache Configuration for Dev/Prod:** Avoid using `@Cacheable` without explicitly defining a `RedisCacheManager` . Falling back to default Spring in-memory `ConcurrentMapCacheManager` in local tests will mask Redis serialization issues until you deploy to production.

## Summary

- `@Cacheable` operates as an interceptor via **Spring AOP proxies**—it executes *before* your method body runs.
- Self-invocation (calling a `@Cacheable` method from within the same class) skips the proxy entirely, causing subtle cache misses.

- Always cache **DTOs** instead of raw **JPA Entities** to avoid proxy serialization failures and memory leaks.
- Use Spring Expression Language (SpEL) to build deterministic composite keys and prevent caching empty/null results.